

Painel de Saúde Pública

Avaliação 2 – Ciência dos Dados

Mariana Lins

NES – Novo Ensino Suplementar
Ciência dos Dados

Junho de 2026



Novo
Ensino
Suplementar



Sumário

- 1 Introdução
- 2 Arquitetura
- 3 Banco de Dados
- 4 Relacionamentos
- 5 API
- 6 Dashboard
- 7 Docker
- 8 Resultados
- 9 Demonstração
- 10 Evidências
- 11 Modelo Preditivo
- 12 Conclusão



Objetivo do Projeto

Desenvolver um sistema de monitoramento de dados de saúde pública utilizando:

- Python
- PostgreSQL
- FastAPI
- Streamlit
- Docker

O sistema permite visualizar indicadores e consultar dados por meio de uma API.



Arquitetura da Solução



Fluxo de comunicação entre Dashboard, API e Banco de Dados.



Tabelas implementadas

- **pacientes**

- paciente_id
- faixa_etaria

- **bairros**

- bairro_id
- nome

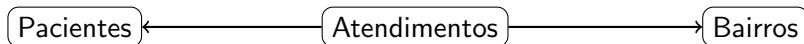
- **atendimentos**

- atendimento_id
- data
- diagnostico
- paciente_id (FK)
- bairro_id (FK)

Os dados foram armazenados em PostgreSQL e populados automaticamente com registros simulados.



Relacionamentos



Cada atendimento está associado a um paciente e a um bairro.



GET /health

- Verifica se a API está ativa.

GET /dados

- Retorna os registros de atendimentos.

GET /resumo

- Retorna indicadores agregados.



Dashboard Streamlit

Funcionalidades implementadas:

- Consumo dos dados da API FastAPI.
- Exibição dos atendimentos cadastrados.
- Indicadores gerais do sistema.
- Integração direta com PostgreSQL via API.
- Interface web acessível pelo navegador.



Containerização

O projeto foi executado utilizando Docker Compose.

Containers utilizados:

- PostgreSQL
- FastAPI
- Streamlit

Benefícios:

- Padronização do ambiente
- Facilidade de execução
- Portabilidade

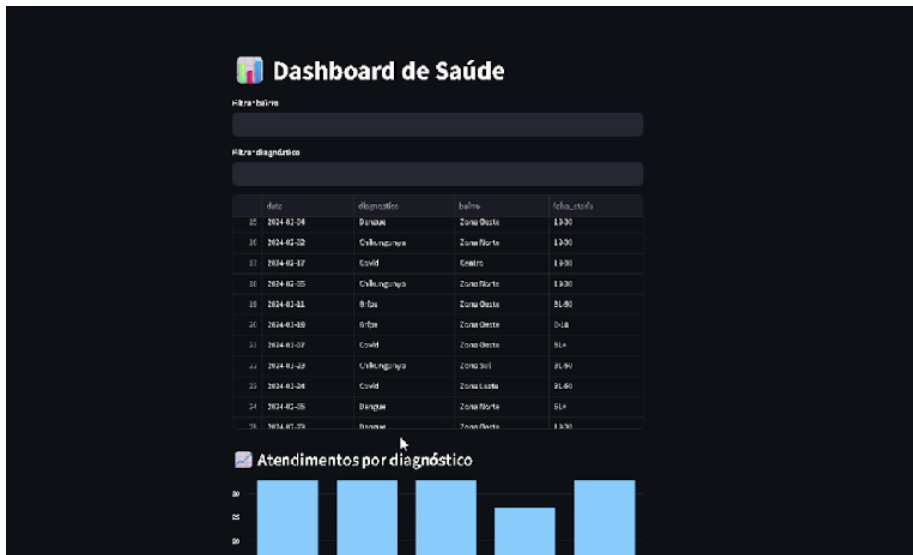


Indicadores do Banco de Dados

- 200 atendimentos gerados
 - 50 pacientes cadastrados
 - 5 bairros cadastrados
 - 5 diagnósticos diferentes
-
- API funcionando corretamente.
 - Dashboard integrado à API.
 - Comunicação entre PostgreSQL, FastAPI e Streamlit validada.
 - Aplicação executada via Docker Compose.



Dashboard - Visão Geral



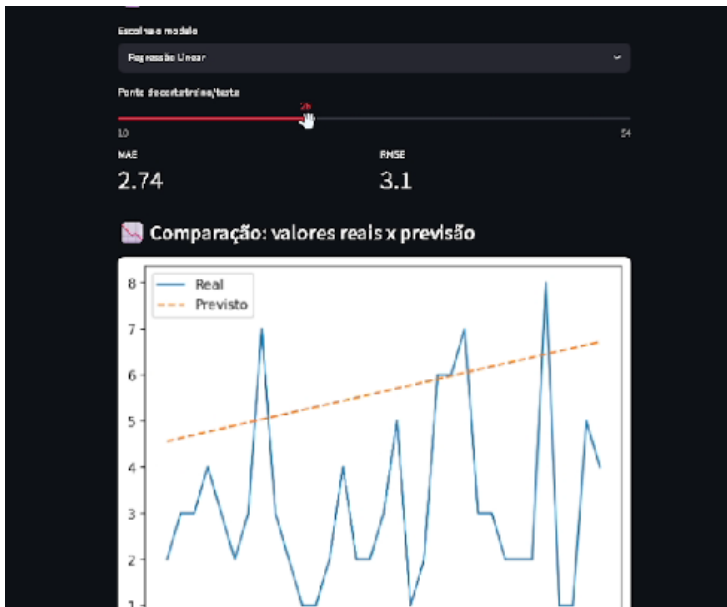
Dashboard - Indicadores



Dashboard - Tabela de Dados



API - Endpoint /health



Validação da API

Endpoints testados:

- GET /health
- GET /dados
- GET /resumo

Resultados observados:

- API respondendo corretamente.
- Banco de dados conectado.
- Dashboard consumindo os dados.
- Containers executando via Docker Compose.



Machine Learning

Foi implementado um módulo de previsão do número de atendimentos diários, permitindo comparar diferentes modelos de Machine Learning e avaliar sua capacidade preditiva.

- Seleção dinâmica do modelo no dashboard;
- Regressão Linear para análise de tendência;
- Random Forest para capturar padrões não lineares;
- Divisão treino/teste configurável pelo usuário;
- Comparação entre valores reais e previstos;
- Métricas MAE e RMSE exibidas em tempo real.



Motivação do Modelo

Objetivo:

Prever o número de atendimentos diários realizados no sistema de saúde pública.

Motivação:

- Auxiliar no planejamento de recursos.
- Antecipar aumentos de demanda.
- Apoiar decisões administrativas.
- Identificar tendências temporais.



Pré-processamento dos Dados

Etapas realizadas:

- Conversão da coluna de datas para formato temporal.
- Agrupamento dos atendimentos por dia.
- Construção da série temporal.
- Divisão dos dados em treino e teste utilizando o ponto de corte t_0 .



Modelos e Hiperparâmetros

Modelos avaliados:

- Regressão Linear
- Random Forest

Configurações utilizadas:

- Random Forest:
 - `n_estimators = 100`
 - `random_state = 42`

Os parâmetros foram definidos por inspeção manual e testes exploratórios.



Métricas e Limitações

Métricas obtidas no período de teste:

- $MAE = 2,74$
- $RMSE = 3,10$

Interpretação:

- Em média, o erro das previsões foi de aproximadamente 2,74 atendimentos por dia.
- O RMSE de 3,10 indica que erros maiores possuem impacto mais significativo na avaliação do modelo.

Limitações:

- Base de dados pequena.
- Dados simulados.
- Não captura sazonalidades complexas.
- Previsões dependem da qualidade histórica dos dados.



Conclusão

Objetivos alcançados:

- Modelagem e armazenamento dos dados em PostgreSQL.
- Desenvolvimento de API REST com FastAPI.
- Criação de dashboard utilizando Streamlit.
- Containerização completa com Docker.
- Integração entre todas as camadas do sistema.

Possíveis melhorias:

- Inclusão de gráficos interativos.
- Atualização em tempo real.
- Novos indicadores epidemiológicos.



Obrigada!

